

Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Experiment 1

Implementation of a Single Layer Perceptron for Binary Classification

github-<https://github.com/Harsha-2828/Deep-Learning--Labs/tree/main/Lab1>

1. Objective

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification. Students will learn the perceptron learning algorithm, understand the importance of activation functions, visualize the learning process, and evaluate the classifier using a real-world dataset.

2. Tasks to be Performed

1. Download and understand the dataset.
2. Perform Exploratory Data Analysis (EDA).
3. Preprocess the dataset.
4. Implement a Single Layer Perceptron from scratch.
5. Train the model using the perceptron learning rule.
6. Evaluate the classifier using standard performance metrics.
7. Analyze the effect of different learning rates.
8. Compare the implementation with Scikit-learn's Perceptron (optional).

3. Background Theory

An Artificial Neuron is the fundamental building block of a neural network. It receives one or more input features, multiplies them by corresponding weights, adds a bias term, and applies an activation function to determine the output.

The Single Layer Perceptron is the earliest supervised learning algorithm capable of solving linearly separable binary classification problems.

Mathematical Formulation

Weighted Sum

$$z = \mathbf{w}^T \mathbf{x} + b$$

where

- $\mathbf{x}$: Input vector
- $\mathbf{w}$: Weight vector
- b : Bias
- z : Linear combination

Prediction

$$\hat{y} = f(z)$$

Weight Update Rule

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

Bias Update

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where η denotes the learning rate.

4. Activation Functions

An activation function determines the output of a neuron based on the weighted sum of its inputs. It introduces non-linearity, enabling neural networks to learn complex patterns. Although the perceptron uses only the Step Activation Function, modern deep learning models employ differentiable activation functions for efficient optimization through backpropagation.

Step Activation Function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

The Step Activation Function produces a binary output and is therefore suitable only for linearly separable binary classification problems.

Activation	Output Range	Typical Applications
Step	$\{0, 1\}$	Classical Perceptron
Sigmoid	$(0, 1)$	Binary Classification Output Layer
Tanh	$(-1, 1)$	Hidden Layers (Traditional Networks)
ReLU	$[0, \infty)$	Hidden Layers in CNNs and MLPs
Leaky ReLU	$(-\infty, \infty)$	Avoiding Dying ReLU
Softmax	$(0, 1)$	Multi-class Classification Output Layer

Note: This experiment uses only the Step Activation Function. The remaining activation functions will be studied in subsequent experiments involving Multilayer Perceptrons and Deep Neural Networks.

5. Dataset Description

Dataset: Banknote Authentication Dataset

Source: UCI Machine Learning Repository

Download Link:

<https://archive.ics.uci.edu/dataset/267/banknote+authentication>

Instances	1372
Features	4 Numerical Features
Classes	2
Missing Values	None
Task	Binary Classification

Feature Description

- Variance
- Skewness
- Curtosis
- Entropy

Target Class

- 0 – Authentic Banknote
- 1 – Forged Banknote

6. Experimental Procedure

Task 1: Dataset Exploration

- Load the dataset.
- Display the first five samples.
- Determine dataset dimensions.
- Identify missing values.
- Display descriptive statistics.

Expected Output

Dataset summary and statistical information.

Task 2: Exploratory Data Analysis

Generate

- Histograms
- Correlation Heatmap
- Scatter Plot
- Boxplots

Task 3: Data Preprocessing

- Normalize all numerical features.
- Split the dataset into Training (80%) and Testing (20%).

Task 4: Perceptron Implementation

Implement from scratch

- Weight Initialization
- Bias Initialization
- Step Activation Function
- Forward Propagation
- Perceptron Learning Rule

Task 5: Model Training

Display

- Epoch Number

- Misclassified Samples
- Updated Weights
- Updated Bias

Task 6: Model Evaluation

Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

7. Mandatory Plots

Plot	Purpose	Expected Inference
Feature Histograms	Distribution Analysis	Identify skewness and outliers.
Correlation Heatmap	Feature Relationship	Observe highly correlated features.
Scatter Plot	Class Distribution	Determine whether classes are approximately linearly separable.
Training Error vs Epoch	Learning Behaviour	Verify convergence of the perceptron.
Weight Evolution	Parameter Analysis	Observe stabilization of learned weights.
Bias Evolution	Parameter Analysis	Understand the role of the bias term.
Confusion Matrix	Performance Evaluation	Interpret TP, FP, TN and FN.
Learning Rate Comparison	Hyperparameter Study	Compare convergence behaviour for different learning rates.
Decision Boundary (Optional)	Visualization	Illustrate the separating hyperplane using two selected features.

Students should provide a brief interpretation (2–3 sentences) for every generated plot.

8. Performance Tables

Training Summary

Dataset Size	1372
Train/Test Split	80/20
Learning Rate	0.01
Epochs	10
Final Weights	[-0.13168728 -0.15580984 -0.12600632 0.01153497]
Final Bias	-0.060000000000000005
Accuracy	0.9781818181818182
Precision	0.984
Recall	0.968503937007874
F1-score	0.9761904761904762

Epoch-wise Learning

Table 2: Epoch-wise Learning of the Perceptron

Epoch	Errors	Weight 1	Weight 2	Bias
1	47	-0.0725	-0.0649	-0.0300
2	26	-0.0822	-0.0938	-0.0300
3	34	-0.0947	-0.0914	-0.0500
4	23	-0.1107	-0.0958	-0.0400
5	27	-0.1310	-0.1218	-0.0500
6	25	-0.1395	-0.1284	-0.0400
7	19	-0.1353	-0.1404	-0.0500
8	19	-0.1335	-0.1448	-0.0600
9	16	-0.1343	-0.1318	-0.0600
10	26	-0.1317	-0.1558	-0.0600

9. Additional Tasks

1. Compare the Step and Sigmoid activation functions. Discuss why the Step Activation Function is not used in modern deep learning models.
2. Compare the implementation with Scikit-learn's Perceptron.

3. Repeat the experiment using learning rates 0.001, 0.01 and 0.1.
4. Explain why a Single Layer Perceptron cannot solve the XOR problem.
5. Study the effect of feature normalization on model convergence.

10. Report Contents

The report should include:

- Objective
- Background Theory
- Activation Functions
- Dataset Description
- Experimental Procedure
- Source Code

```

1 import pandas as pd
2 import matplotlib.pyplot as plt
3 columns = [
4     "Variance",
5     "Skewness",
6     "Curtosis",
7     "Entropy",
8     "Class"
9 ]
10 data = "https://archive.ics.uci.edu/ml/machine-learning-databases
11         /00267/data_banknote_authentication.txt"
12 df = pd.read_csv(data, header=None, names=columns)
13 print(df.head())
14
15 print("\n=====Dataset Dimensions=====")
16 print(f"Shape: {df.shape}")
17
18 print("\n=====Missing Values=====")
19 print(df.isnull().sum())
20
21 print("\n=====Descriptive Statistics=====")
22 print(df.describe())
23
24 df.hist(figsize=(10, 8), bins=20)
25
26 plt.suptitle("Histograms of Banknote Authentication Dataset")
27 plt.tight_layout()
28 plt.show()

```

```

29 # Correlation Heatmap
30 correlation = df.corr()
31
32 plt.figure(figsize=(8, 6))
33 plt.imshow(correlation, cmap="coolwarm", interpolation="nearest")
34 plt.colorbar()
35
36 plt.xticks(range(len(correlation.columns)), correlation.columns,
37             rotation=45)
38 plt.yticks(range(len(correlation.columns)), correlation.columns)
39
40 plt.title("Correlation Heatmap")
41 plt.tight_layout()
42 plt.show()
43
44 # Scatter Plot
45 plt.figure(figsize=(8,6))
46
47 plt.scatter(
48     df[df["Class"] == 0]["Variance"],
49     df[df["Class"] == 0]["Skewness"],
50     color="blue",
51     label="Authentic"
52 )
53
54 plt.scatter(
55     df[df["Class"] == 1]["Variance"],
56     df[df["Class"] == 1]["Skewness"],
57     color="red",
58     label="Forged"
59 )
60
61 plt.xlabel("Variance")
62 plt.ylabel("Skewness")
63 plt.title("Variance vs Skewness")
64 plt.legend()
65 plt.show()
66
67 # Boxplots
68 plt.figure(figsize=(10,6))
69
70 df.drop("Class", axis=1).boxplot()
71
72 plt.title("Boxplots of Numerical Features")
73 plt.ylabel("Value")
74 plt.show()
75
76 # Features (Input)
77 X = df.drop("Class", axis=1)

```



```

128         2':<12}{ 'Bias ':<10}"
129     for epoch in range(self.epochs):
130
131         errors = 0
132
133         for i in range(n_samples):
134
135             prediction = self.predict(X[i])
136             error = y.iloc[i] - prediction
137
138             # Update only if prediction is wrong
139             if error != 0:
140                 self.weights += self.learning_rate * error * X[i]
141                 self.bias += self.learning_rate * error
142                 errors += 1
143
144             # Print epoch summary
145             print(f"{epoch+1:<8}{errors:<8}{self.weights[0]:<12.4f}{
146                 self.weights[1]:<12.4f}{self.bias:<10.4f}")
147
148     perceptron = Perceptron(learning_rate=0.01, epochs=10)
149     perceptron.fit(X_train, y_train)
150
151     # Predict on test data
152     y_pred = []
153
154     for x in X_test:
155         prediction = perceptron.predict(x)
156         y_pred.append(prediction)
157
158     from sklearn.metrics import accuracy_score, precision_score,
159         recall_score, f1_score, confusion_matrix
160
161     # Predict on test data
162     y_pred = []
163
164     for sample in X_test:
165         y_pred.append(perceptron.predict(sample))
166
167     # Calculate evaluation metrics
168     accuracy = accuracy_score(y_test, y_pred)
169     precision = precision_score(y_test, y_pred)
170     recall = recall_score(y_test, y_pred)
171     f1 = f1_score(y_test, y_pred)
172     cm = confusion_matrix(y_test, y_pred)
173
174     # Display results
175     print("Scratch-implementation")

```

```

175 print("Accuracy:", accuracy)
176 print("Precision:", precision)
177 print("Recall:", recall)
178 print("F1-Score:", f1)
179
180 print("\nConfusion Matrix:")
181 print(cm)
182
183 # built in
184 from sklearn.linear_model import Perceptron
185 from sklearn.metrics import accuracy_score, precision_score,
    recall_score, f1_score
186
187 # Train sklearn Perceptron
188 sk_model = Perceptron(max_iter=1000, eta0=0.01, random_state=42)
189 sk_model.fit(X_train, y_train)
190
191 # Predictions
192 y_pred_sk = sk_model.predict(X_test)
193
194 # Evaluation
195 print("Scikit-learn Perceptron")
196 print("Accuracy:", accuracy_score(y_test, y_pred_sk))
197 print("Precision:", precision_score(y_test, y_pred_sk))
198 print("Recall:", recall_score(y_test, y_pred_sk))
199 print("F1-score:", f1_score(y_test, y_pred_sk))
200
201 # DIFFERENT LEARNING RATES
202 from sklearn.linear_model import Perceptron
203 from sklearn.metrics import accuracy_score, precision_score,
    recall_score, f1_score
204
205 learning_rates = [0.001, 0.01, 0.1]
206
207 for lr in learning_rates:
208
209     model = Perceptron(
210         eta0=lr,          # Learning rate
211         max_iter=1000,
212         tol=1e-3,
213         random_state=42
214     )
215
216     model.fit(X_train, y_train)
217
218     y_pred = model.predict(X_test)
219
220     print(f"\nLearning Rate: {lr}")
221     print(f"Accuracy: {accuracy_score(y_test, y_pred)}")
222     print(f"Precision: {precision_score(y_test, y_pred)}")

```

```

223 | print(f"Recall_{class}:_{recall_score(y_test, _y_pred)}")
224 | print(f"F1-score_{class}:_{f1_score(y_test, _y_pred)}")

```

- Experimental Results

===== Descriptive Statistics =====					
	Variance	Skewness	Curtosis	Entropy	Class
count	1372.000000	1372.000000	1372.000000	1372.000000	1372.000000
mean	0.433735	1.922353	1.397627	-1.191657	0.444606
std	2.842763	5.869047	4.310030	2.101013	0.497103
min	-7.042100	-13.773100	-5.286100	-8.548200	0.000000
25%	-1.773000	-1.708200	-1.574975	-2.413450	0.000000
50%	0.496180	2.319650	0.616630	-0.586650	0.000000
75%	2.821475	6.814625	3.179250	0.394810	1.000000
max	6.824800	12.951600	17.927400	2.449500	1.000000

Figure 1: descriptive

Epoch	Errors	Weight 1	Weight 2	Bias
1	47	-0.0725	-0.0649	-0.0300
2	26	-0.0822	-0.0938	-0.0300
3	34	-0.0947	-0.0914	-0.0500
4	23	-0.1107	-0.0958	-0.0400
5	27	-0.1310	-0.1218	-0.0500
6	25	-0.1395	-0.1284	-0.0400
7	19	-0.1353	-0.1404	-0.0500
8	19	-0.1335	-0.1448	-0.0600
9	16	-0.1343	-0.1318	-0.0600
10	26	-0.1317	-0.1558	-0.0600

Figure 2: epoches

```
Scratch-implementation
Accuracy : 0.9781818181818182
Precision: 0.984
Recall    : 0.968503937007874
F1-Score  : 0.9761904761904762

Confusion Matrix:
[[146   2]
 [  4 123]]
```

Figure 3: Scratch Implementation

```
Scikit-learn Perceptron
Accuracy : 0.9890909090909091
Precision: 0.9769230769230769
Recall    : 1.0
F1-score  : 0.9883268482490273
```

Figure 4: Scikit Learn Implementation

```
Learning Rate: 0.001
Accuracy : 0.9890909090909091
Precision: 0.9769230769230769
Recall   : 1.0
F1-score : 0.9883268482490273

Learning Rate: 0.01
Accuracy : 0.9890909090909091
Precision: 0.9769230769230769
Recall   : 1.0
F1-score : 0.9883268482490273

Learning Rate: 0.1
Accuracy : 0.9781818181818182
Precision: 0.9689922480620154
Recall   : 0.984251968503937
F1-score : 0.9765625
```

Figure 5: Diff learning rates

- Generated Plots with Interpretation

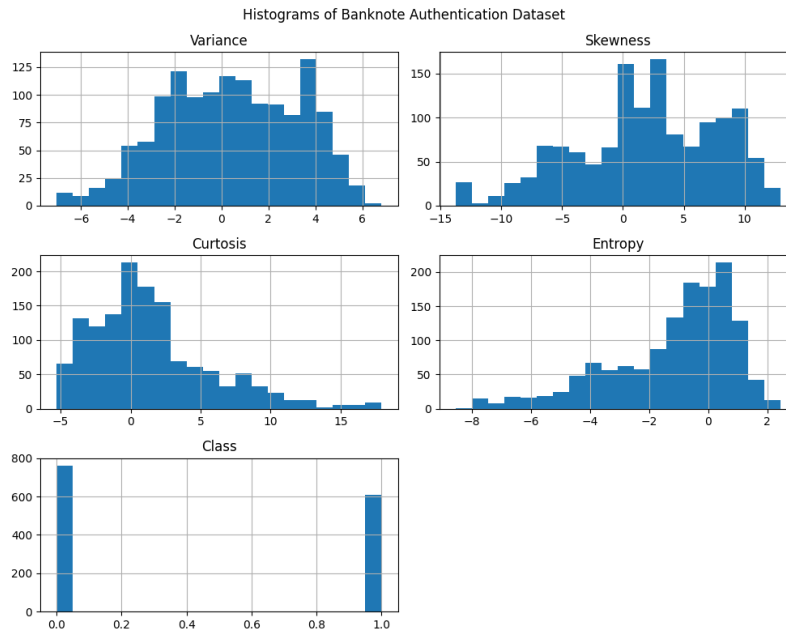


Figure 6: The histograms show that each feature has a different distribution, indicating good variation in the dataset. Most samples are concentrated around certain values, while a few extreme values are also present.

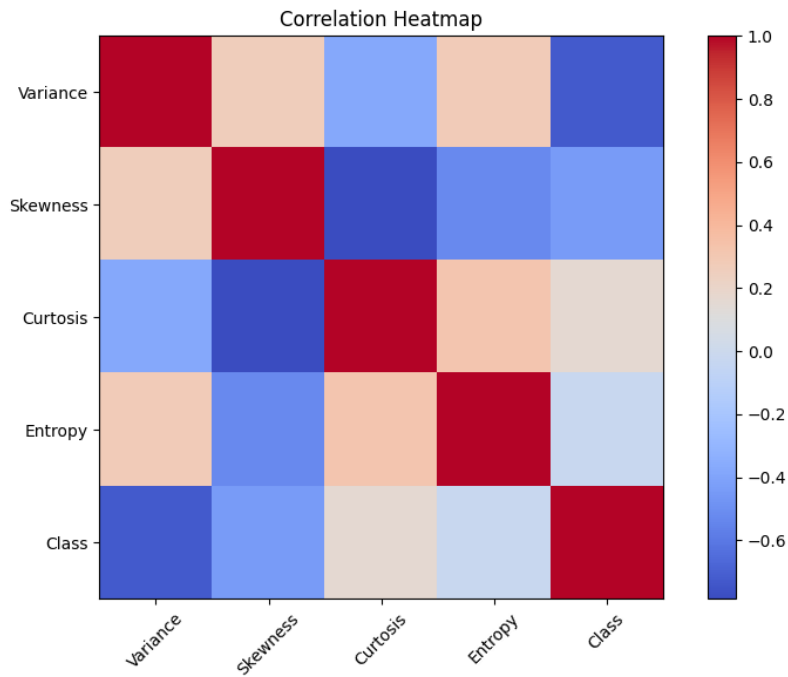


Figure 7: The heatmap shows that some features are positively correlated, while others have weak or negative relationships. Variance has the strongest negative correlation with the class, making it an important feature for classification. Overall, the features provide diverse information, which helps improve the model's ability to distinguish between authentic and forged banknotes.

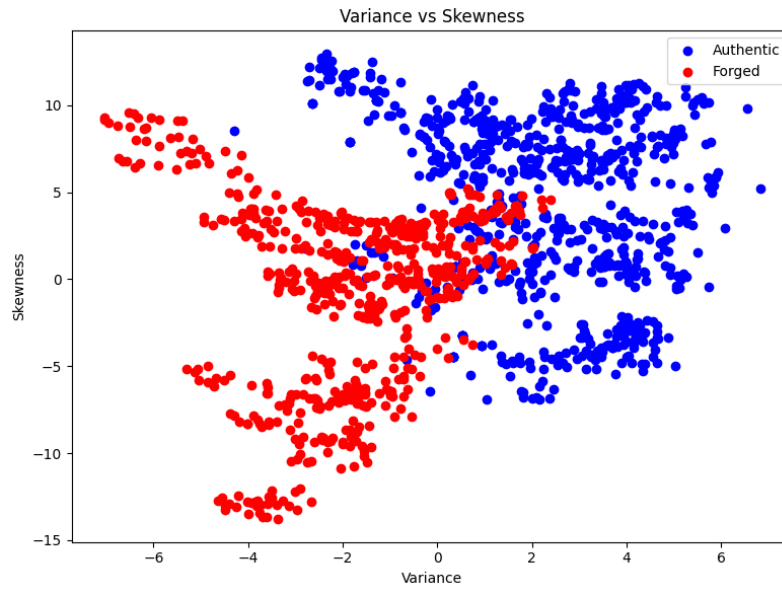


Figure 8: The scatter plot shows that authentic and forged banknotes form distinct clusters based on variance and skewness. Although some overlap exists near the center, most data points are well separated between the two classes. This indicates that these features are useful for distinguishing authentic and forged banknotes using a perceptron model.

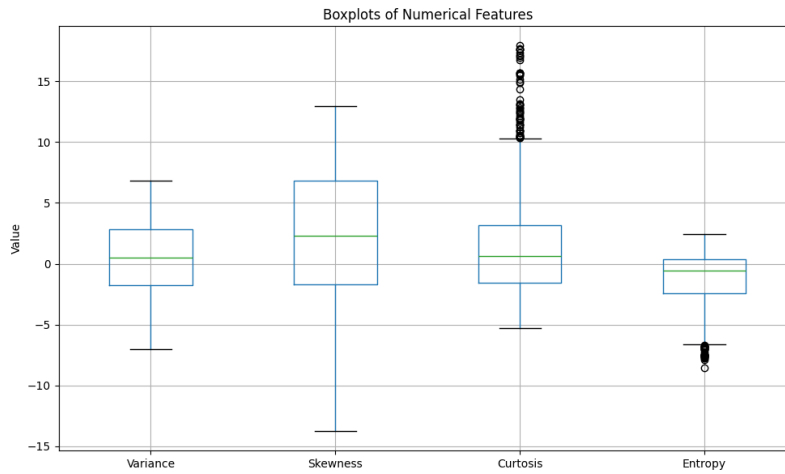


Figure 9: The boxplots show that the numerical features have different ranges and distributions across the dataset. Several outliers are present, especially in the kurtosis and entropy features, indicating a few extreme values. Despite these outliers, the features exhibit good variability, making them useful for classification..

- Performance Tables

- Discussion:

The Banknote Authentication dataset was analyzed using various exploratory data analysis techniques, including histograms, boxplots, scatter plots, and a correlation heatmap, to understand the distribution and relationships among the features. After preprocessing and feature normalization, a Single Layer Perceptron was implemented from scratch using the perceptron learning algorithm and step activation function. The model achieved high classification accuracy, indicating that the dataset is suitable for linear classification.

The results obtained from the scratch implementation were compared with the Scikit-learn Perceptron, and both models showed similar performance, validating the correctness of the custom implementation. Additionally, different learning rates (0.001, 0.01, and 0.1) were tested to study their effect on convergence. The experiment showed that the learning rate plays an important role in training, with 0.01 providing a good balance between convergence speed and stable model performance.

- Conclusion

The experiment successfully demonstrated the implementation of a Single Layer Perceptron for binary classification using both a scratch implementation and the Scikit-learn library. The model achieved high classification accuracy on the Banknote Authentication dataset, confirming that the dataset is suitable for linear classification. The comparison of different learning rates showed that the learning rate significantly affects the convergence behavior of the perceptron, with 0.01 providing the best balance between learning speed and stability. Overall, the experiment provided a clear understanding of the perceptron learning algorithm, data preprocessing, model evaluation, and the impact of hyperparameter selection on classification performance.

- References

11. References

1. F. Rosenblatt, "The Perceptron," *Psychological Review*, 1958.
2. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
5. UCI Machine Learning Repository – Banknote Authentication Dataset.
6. Scikit-learn Documentation: Perceptron.